📝 Problem Set

Task 1: Build an AI Assistant (Applied AI)

Objective

Develop a robust AI assistant utilizing modern LLM APIs and RAG architectures.

Core Functionality

- **LLM Integration**: Connect to a major provider (OpenAI, Gemini, Claude, or Bedrock).
- **Prompt Engineering**: Design system prompts and tune parameters like *temperature* and *top_p*.
- **Structured Output**: Ensure the model generates valid **JSON** responses.
- **Tool Calling**: Implement function calling to allow the AI to interact with external tools.

Technical Implementation

- **RAG Pipeline**: Build a Retrieval-Augmented Generation system including:
    - Efficient **Document Ingestion** and **Chunking**.
    - Vectorization using **Embeddings** stored in a **Vector Database**.
- **Local Deployment**: Serve an open-source model (e.g., Llama 3, Mistral) locally using **vLLM**.
- **Containerization**: Package the entire application using **Docker**.

Deliverables

- Source code
- Dockerfile
- README
- Architecture diagram

Task 2: Productionize the AI Assistant (Engineering AI Systems)

Objective

Transform the model you trained in the past sessions into a production-ready application.

Requirements

Application

- Build a simple web UI (e.g., Streamlit, Gradio, React, etc.)
- Connect the UI to the AI backend

Model Optimization *(optional where applicable)*

- Convert the model to ONNX (or justify why not applicable)
- Apply inference optimizations if supported

Performance Engineering

- Handle concurrent/batch request processing or asynchronous requests handling
- Optimize latency and throughput
- Implement prompt/response caching (optional bonus)

Reliability

Implement:

- Retry mechanism
- Rate limiting
- Fallback model/provider
- Error handling & graceful degradation

Deployment

- Dockerize the complete application
- Provide deployment instructions
- *(Bonus: Deploy to Azure, AWS, GCP)*

Deliverables

- Updated source code
- Docker Compose configuration
- Architecture diagram